Jawaban Soal Teori

Function, Parameter, Return Value, Scope, Lambda, Module, Package & Library

NAMA	:	**EKA FEBRYANTO**
NIM	:	**20241320014**
KELAS	:	**A1 SISTEM INFORMASI**

SOAL 1

> Jelaskan perbedaan antara parameter dan argumen. Berikan contoh masing-masing dalam konteks fungsi Python.

Parameter adalah nama variabel yang dideklarasikan pada definisi fungsi (di dalam tanda kurung setelah nama fungsi). Parameter berfungsi sebagai *placeholder* atau wadah yang menunjukkan data apa yang diharapkan fungsi. **Argumen** adalah nilai nyata yang diberikan kepada fungsi pada saat fungsi tersebut dipanggil (*function call*). Nilai argumen inilah yang kemudian terikat ke parameter.

```
def hitung_luas(panjang, lebar):  # panjang, lebar = PARAMETER
    return panjang * lebar
hasil = hitung_luas(5, 3)  # 5, 3 = ARGUMEN
print(hasil)  # 15
```

Aspek	Parameter	Argumen
Lokasi	Pada definisi fungsi (def)	Pada pemanggilan fungsi
Sifat	Nama variabel / placeholder	Nilai atau ekspresi aktual
Contoh	panjang, lebar	5, 3

Jenis-jenis Parameter dan Argumen

- **Positional dan Keyword**: argumen dicocokkan berdasarkan urutan, atau disebutkan dengan nama parameternya.
- **Default**: parameter memiliki nilai bawaan yang dipakai jika argumen tidak diberikan.
- **Variabel** (*args dan **kwargs): menampung sejumlah argumen positional dan keyword yang tidak tentu banyaknya.

```
def perkenalan(nama, kota="Bandung", *hobi, **info):
    print(nama, kota, hobi, info)
perkenalan("Rani")  # positional + default
perkenalan(kota="Surabaya", nama="Dimas")  # keyword
perkenalan("Rani", "Jakarta", "membaca", umur=20)  # *args dan **kwargs
```

SOAL 2

> Apa yang dimaksud dengan aturan LEGB dalam scope Python? Jelaskan setiap level dan berikan contoh situasi di mana variabel dari masing-masing level diakses.

Scope adalah wilayah dalam program tempat sebuah nama (variabel) dapat dikenali dan diakses. **Aturan LEGB** adalah urutan pencarian yang dilakukan Python ketika menemukan sebuah nama variabel. Python mencari dari lingkup paling dalam ke paling luar, dan berhenti begitu nama ditemukan. LEGB merupakan singkatan dari **L**ocal, **E**nclosing, **G**lobal, dan **B**uilt-in.

Urutan	Level	Cakupan	Contoh Situasi Akses
1	**L**ocal	Variabel yang dibuat di dalam fungsi yang sedang dieksekusi.	Variabel penampung hasil sementara dalam sebuah fungsi.
2	**E**nclosing	Variabel pada fungsi luar yang membungkus fungsi dalam (*nested function*).	Fungsi dalam / closure membaca variabel milik fungsi pembungkusnya.
3	**G**lobal	Variabel pada level modul (file .py), di luar semua fungsi.	Konstanta atau konfigurasi yang dipakai oleh banyak fungsi.
4	**B**uilt-in	Nama bawaan Python yang tersedia otomatis.	Pemanggilan len(), print(), sum(), range().

Contoh Kode Gabungan Keempat Level

```
x = "global"  # G - Global

def luar():
    x = "enclosing"  # E - Enclosing (milik fungsi luar)

    def dalam():
        x = "local"  # L - Local (milik fungsi dalam)
        print(x)  # -> local

    dalam()
    print(x)  # -> enclosing

luar()
print(x)  # -> global
print(len("LEGB"))  # B - Built-in -> 4
```

Penjelasan Tiap Level

1. Local. Variabel yang dibuat di dalam fungsi hanya hidup selama fungsi dijalankan dan tidak dapat diakses dari luar. Mengaksesnya dari luar akan menimbulkan NameError.

```
def hitung_diskon(harga):
    diskon = harga * 0.1  # variabel local
    return harga - diskon

print(hitung_diskon(100000))  # 90000.0
# print(diskon)  # NameError: name 'diskon' is not defined
```

2. Enclosing. Terjadi pada fungsi bersarang. Fungsi dalam dapat membaca variabel milik fungsi luar. Untuk *mengubah* variabelnya, gunakan kata kunci nonlocal.

```
def buat_penghitung():
    hitung = 0  # variabel enclosing

    def tambah():
        nonlocal hitung  # modifikasi var. enclosing
        hitung += 1
        return hitung

    return tambah

pc = buat_penghitung()
print(pc(), pc(), pc())  # 1 2 3
```

3. Global. Variabel yang didefinisikan di level modul dapat dibaca dari fungsi mana pun. Untuk *mengubahnya* dari dalam fungsi, gunakan kata kunci global.

```
PPN = 0.11  # variabel global

def harga_akhir(harga):
    return round(harga * (1 + PPN))  # membaca variabel global

total = 0
def tambah_total(n):
    global total  # modifikasi var. global
    total += n

tambah_total(50)
print(harga_akhir(100000), total)  # 111000 50
```

4. Built-in. Lingkup terluar berisi nama-nama yang disediakan Python tanpa perlu import. Python hanya akan mencarinya di sini jika nama tidak ditemukan di tiga level sebelumnya. Karena itu, menamai variabel dengan nama built-in (misalnya list atau sum) akan *menimpa* (*shadowing*) fungsi bawaan tersebut dan sebaiknya dihindari.

```
data = [3, 1, 2]
print(len(data), max(data), sorted(data))  # semua diambil dari scope Built-in
```

SOAL 3

> Mengapa penggunaan objek mutable sebagai default parameter dianggap sebagai pitfall? Jelaskan mekanisme di balik perilaku ini dan berikan solusi yang benar.

Masalahnya

Objek *mutable* (misalnya list, dict, dan set) yang dipakai sebagai nilai default parameter akan **dipakai bersama oleh seluruh pemanggilan fungsi**. Akibatnya, perubahan yang dilakukan pada satu pemanggilan akan terbawa ke pemanggilan berikutnya, sehingga menghasilkan perilaku yang tidak terduga.

```
def tambah_item(item, daftar=[]):  # default list mutable
    daftar.append(item)
    return daftar

print(tambah_item("apel"))  # ['apel']
print(tambah_item("jeruk"))  # ['apel', 'jeruk'] <- tak terduga
print(tambah_item("mangga"))  # ['apel', 'jeruk', 'mangga']
```

Mekanisme di Balik Perilaku Ini

Nilai default parameter **dievaluasi hanya satu kali, yaitu saat pernyataan def dijalankan** (saat fungsi didefinisikan), bukan setiap kali fungsi dipanggil. Prosesnya sebagai berikut:

1. Ketika Python mengeksekusi def, ekspresi [] dievaluasi dan satu objek list dibuat di memori.
2. Objek tersebut disimpan di atribut fungsi, yaitu tambah_item.__defaults__, yang kemudian berisi (['apel', 'jeruk', 'mangga'],).
3. Setiap pemanggilan tanpa argumen daftar akan memakai **objek list yang sama** tersebut.
4. Karena list bersifat mutable, append() mengubah isi objek yang sama sehingga perubahan tersimpan antar-pemanggilan.

Hal ini tidak terjadi pada objek *immutable* (seperti int, str, tuple, dan None) karena nilainya tidak dapat diubah di tempat. Operasi seperti += pada int membuat objek baru, bukan memodifikasi objek default.

Solusi yang Benar

Gunakan None sebagai nilai default (*sentinel value*), lalu buat objek mutable yang baru **di dalam badan fungsi**. Dengan cara ini, setiap pemanggilan mendapat objek baru yang independen.

```
def tambah_item(item, daftar=None):
    if daftar is None:  # dibuat baru pada SETIAP pemanggilan
        daftar = []
    daftar.append(item)
    return daftar

print(tambah_item("apel"))  # ['apel']
print(tambah_item("jeruk"))  # ['jeruk'] <- benar
```

Gunakan pemeriksaan is None (bukan if not daftar) agar list kosong yang sengaja diberikan pemanggil tetap dihormati dan tidak ikut digantikan.

SOAL 4

> Jelaskan perbedaan antara module dan package dalam Python. Apa peran file __init__.py dalam sebuah package?

Module adalah sebuah file Python tunggal berekstensi .py yang berisi definisi fungsi, kelas, dan variabel yang dapat digunakan ulang melalui import. **Package** adalah sebuah direktori yang berisi satu atau lebih module (dan dapat pula subpackage), sehingga berfungsi untuk mengelompokkan module yang saling berkaitan secara hierarkis.

Aspek	Module	Package
Bentuk	Satu file .py	Direktori berisi kumpulan module
Fungsi	Mengelompokkan kode terkait dalam satu file	Mengorganisasi banyak module dalam struktur hierarkis
Contoh	math, random, utils.py	numpy, pandas, django
Cara import	import utils	import paket.modul atau from paket import modul

Contoh Struktur

```
proyek/
  main.py
  toko/   (package)
    __init__.py
    produk.py   (module)
    pembayaran.py   (module)
    laporan/   (subpackage)
      __init__.py
      harian.py
```

```
# main.py
import toko.produk
from toko.pembayaran import hitung_total
from toko.laporan import harian
```

Peran File __init__.py

File __init__.py berada di dalam direktori package dan memiliki beberapa peran penting:

- **Penanda package.** Keberadaannya menandai bahwa sebuah direktori diperlakukan sebagai *regular package* oleh Python. (Sejak Python 3.3 terdapat *namespace package* yang tidak memerlukan file ini, tetapi __init__.py tetap dianjurkan.)
- **Kode inisialisasi.** Isinya dieksekusi otomatis satu kali saat package pertama kali di-import, misalnya untuk menyiapkan konfigurasi atau variabel tingkat package.

- **Menyederhanakan API.** Dapat meng-*import* ulang nama-nama penting dari submodule agar pengguna cukup mengimpor dari package-nya saja.
- **Mengontrol wildcard import.** Variabel __all__ menentukan nama apa saja yang diekspor ketika pengguna menulis from paket import *.

```
# toko/__init__.py
from .produk import Produk
from .pembayaran import hitung_total

__all__ = ["Produk", "hitung_total"]
__version__ = "1.0"

# Akibatnya pengguna cukup menulis:
#   from toko import Produk, hitung_total
# bukan:
#   from toko.produk import Produk
```

SOAL 5

> Kapan sebaiknya menggunakan lambda alih-alih def? Sebutkan setidaknya tiga skenario di mana lambda lebih tepat, dan tiga skenario di mana def lebih tepat.

lambda adalah cara membuat **fungsi anonim** satu baris yang hanya berisi satu *ekspresi* dan otomatis mengembalikan hasil ekspresi tersebut. def digunakan untuk membuat fungsi bernama yang dapat berisi banyak pernyataan. Pilihan di antara keduanya bergantung pada kompleksitas dan kebutuhan keterbacaan kode.

```
def kuadrat(x):
    return x ** 2

kuadrat_lambda = lambda x: x ** 2  # ekuivalen (contoh saja)
```

A. Skenario di Mana lambda Lebih Tepat

1. Fungsi kunci pada sorted(), min(), max(). Fungsi sederhana yang hanya dipakai sekali untuk menentukan kriteria pengurutan.

```
mahasiswa = [("Ani", 85), ("Budi", 92), ("Citra", 78)]
print(sorted(mahasiswa, key=lambda m: m[1], reverse=True))
# [('Budi', 92), ('Ani', 85), ('Citra', 78)]
```

2. Argumen untuk map(), filter(), dan reduce(). Transformasi atau penyaringan singkat yang cukup ditulis di tempat.

```
angka = [1, 2, 3, 4, 5, 6]
genap   = list(filter(lambda n: n % 2 == 0, angka))  # [2, 4, 6]
dikali3 = list(map(lambda n: n * 3, angka))  # [3, 6, ..., 18]
```

3. Callback singkat atau fungsi sekali pakai. Misalnya sebagai handler pada tombol GUI, atau sebagai nilai dalam dictionary yang memetakan operasi.

```
operasi = {
    "tambah": lambda a, b: a + b,
    "kali":   lambda a, b: a * b,
}
print(operasi["kali"](4, 5))  # 20
```

B. Skenario di Mana def Lebih Tepat

1. Logika lebih dari satu ekspresi. Lambda tidak dapat memuat beberapa pernyataan, seperti perulangan, try/except, atau assignment bertahap. Logika yang panjang atau bercabang harus memakai def.

```
def klasifikasi_nilai(n):
    if n >= 85:
        return "A"
    elif n >= 70:
        return "B"
    return "C"
```

2. Fungsi yang dipakai ulang di banyak tempat. Fungsi bernama lebih mudah dipanggil, diuji (*unit test*), dan dipelihara. Lambda yang di-*assign* ke variabel justru dianggap praktik kurang baik menurut PEP 8.

3. Memerlukan dokumentasi dan keterbacaan. def mendukung *docstring*, *type hint*, dan nama yang jelas pada traceback saat terjadi error. Lambda selalu tampil sebagai <lambda> di traceback sehingga menyulitkan *debugging*.

```
def hitung_bmi(berat: float, tinggi: float) -> float:
    """Menghitung Body Mass Index dari berat (kg) dan tinggi (m)."""
    if tinggi <= 0:
        raise ValueError("Tinggi harus lebih dari 0")
    return berat / tinggi ** 2
```

Ringkasan Perbandingan

Aspek	lambda	def
Nama fungsi	Anonim	Bernama
Isi	Satu ekspresi saja	Banyak pernyataan
Docstring / type hint	Tidak didukung	Didukung
Penggunaan ideal	Fungsi pendek sekali pakai (key, map, filter, callback)	Logika kompleks, dipakai ulang, perlu diuji dan didokumentasikan